浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

Python II: Working with Numbers

Basic Data Types

IBI 1, Lecture 6.1

Ying CHI – yingchi.intl.zju.edu.cn

Adapted from Rob

Semester 2, 2025/26

How do you tell a computer what to do?

When do biomedical informaticians need numbers?

When do biomedical informaticians need numbers?

Table 1. Results of coexpression analysis for a range of human traits for which high-quality data are available: Crohn's disease, ulcerative colitis, high-density lipoprotein (HDL), low-density lipoprotein (LDL), total cholesterol, triglycerides, height, systolic blood pressure (SBP) and diastolic blood pressure (DBP).

Trait	SNPs included, p<5e-6 (SNPs per million bases)	Regulatory regions containing a SNP (SNPs per million bases)	Fold enrichment for SNPs in regulatory regions	Distinct regulatory regions	Significantly coexpressed TSS (FDR<0.05)(% of distinct regions)	p (KS test)
Crohn's disease	1924 (0.6)	133 (3.5)	5.7	70	23 (33%)	1.61e-05
Ulcerative colitis	2162 (0.7)	146 (3.8)	5.5	83	20 (24%)	2.28e-06
LDL	4644 (1.5)	205 (5.2)	3.5	92	19 (21%)	1.48e-04
Total cholesterol	6421 (2.0)	316 (8.3)	4.1	128	29 (23%)	6.55e-07
Triglycerides	4863 (1.5)	254 (7.0)	4.6	97	23 (24%)	8.35e-06
Height	8882 (2.8)	358 (7.6)	2.7	166	29 (17%)	1.25e-06
HDL	5410 (1.7)	260 (7.2)	4.2	101	17 (17%)	3.51e-04
SBP	417 (0.1)	20 (0.4)	3.0	13	0 (0%)	4.89e-01
DBP	711 (0.2)	20 (0.4)	1.9	14	0 (0%)	5.41e-01

Date

Learning objectives

After this lecture, you should be able to:

- Name and describe data types (revision).
- Manipulate numbers in Python.
- Visualise and summarise data in Python.

This lecture is a Python reference guide

"They're encyclopedias, Timmy. . . they're an early form of

- The information here will help you write code, but its best to practice – not to memorise.
- You can often achieve the same thing by different means, and you should choose what works best for you (not me!).

Outline

1. Variable types
2. Expressions
3. Simple operations – maths/comparison/logic
4. Data Visualisation – matplotlib

Variables

- Programs manipulate **variables** (see lecture 5.1).

- Variables have a **type** that defines the kinds of things programs can do to them.
  - Anna is a human, so she can walk, speak English, etc.
  - Chewbacca is a wookie so he can walk, "mwaaarhrhh", etc.

- Objects are:
  - scalar (cannot be subdivided)
  - non-scalar (have internal structure that can be accessed)

Scalar variables

- `Int` – represents **integers**, e.g. 5
- `Float` – represents **real numbers**, e.g. 3.27.
- `Boolean` – represents **Boolean** values True and False.
- `Complex` – represents **complex numbers**, e.g. 2+-3j/2+-3j
- Can use `type()` to see the type of an object.

```
In [1]: type(5)
Out[1]: int

In [2]: type(3.0)
Out[2]: float
```

Type conversations

- Python can **convert objects of one type to another**
- `float(3)` converts integer 3 to float `3.0`.
- `int(3.9)` truncates float `3.9` to integer 3.
- What would be the output of the following commands: `float(15)`, `int(6.1)`, `int(15)`, `float(6.1)`?

Printing to console

- To show outputs from code to a user, use the `print` command.

```
In [1]: 3+2
Out[1]: 5
```

"Out" tells you it's an interaction with the shell only

```
In [2]: print(3+2)
5
```

No "Out" means it is actually shown to a user, apparent when you edit/run files

Outline

1. Variable types
2. Expressions
3. Simple operations – maths/comparison/logic
4. Data Visualisation – matplotlib

Expressions

- You can **combine objects and operators** to form expressions.
- An expression has a **value**, which has a type.
- syntax for a simple expression:

```
<object> <operator> <object>
```

Number operations: review

- `i+j` → the **sum**
- `i-j` → the **difference**
- `i*j` → the **product**
- `i/j` → **division**
- `i%j` → the **remainder** when `i` is divided by `j`
- `i**j` → `i` to the **power** of `j`

Number operations

- `i+j` → the **sum**
- `i-j` → the **difference**
- `i*j` → the **product**

if both are ints, result is int
if either or both are floats, result is float

- `i/j` → **division** → result is float
- `i%j` → the **remainder** when `i` is divided by `j`
- `i**j` → `i` to the **power** of `j`

Number operations

- `i+j` → the **sum**
- `i-j` → the **difference**
- `i*j` → the **product**
- `i/j` → **division** → result is
- `i%j` → the **remainder** when `i` is divided by `j`
- `i**j` → `i` to the **power** of `j`

s, result is int
re floats,

Just try one example and see if you get sensible output!!

Outline

1. Variable types
2. Expressions
3. Simple operations – maths/comparison/logic
4. Data Visualisation – matplotlib

Simple operations

- Parentheses ( ) used to tell Python to do these operations first.
- Operator precedence without parentheses.
- **
- *
- /
- + and – executed left to right, as they appear in the expression

Simple operations

- Parentheses ( ) used to tell Python to do [illegible] st.
- Operator precedence without pare[illegible]
- **
- *
- /
- + and – executed left to right, as they appear in the expression

Just try one example and see if you get sensible output!!

Comparing values: review

- Booleans are often used for comparison.
- Is a smaller than b?

  `a < b`
- Is a smaller than or equal to b?

  `a <= b`
- Is a greater than or equal to b?

  `a >= b`
- Is a equal to b?

  `a == b`

  Why two equal signs?
- Is a not equal to b?

  `a != b`

Truth tables: review

Booleans can be combined and modified using not, and, and or. If A is True, then not A is False, and the other way round. The meanings of and and or can be seen from these truth tables.

And		
A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Or		
A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Logic example

```
current_time = 22
bed_time = 20
print(bed_time > current_time)
```

Logic example

```
current_time = 22
bed_time = 20
print(bed_time > current_time) # evaluates to "False"
Exercise = True
drink = False
both = drink and exercise
```

Logic example

```
current_time = 22
bed_time = 20
print(bed_time > current_time) # evaluates to "False"
Exercise = True
drink = False
both = drink and exercise # evaluates to "False"
print(both)
```

Outline

1. Variable types
2. Expressions
3. Simple operations – maths/comparison/logic
4. Data Visualisation – matplotlib

Data visualisation

Matplotlib is probably the most used Python package for 2D graphics.

Before using, remember to import modules:

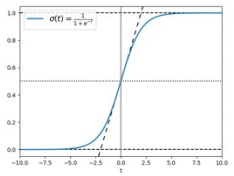
```
import matplotlib as plt
import numpy as np
```

More usage about matplotlib:

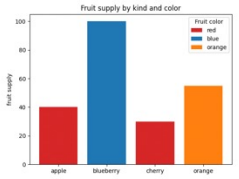
https://matplotlib.org/gallery/index.html
http://www.labri.fr/perso/nrougier/teaching/matplotlib/

Some examples

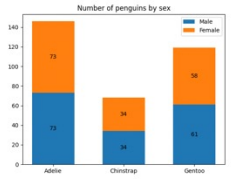
Lines, bars and markers



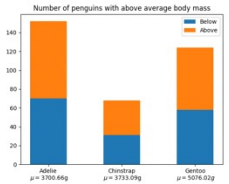
Infinite lines



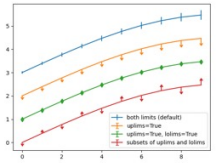
Bar chart with individual bar colors



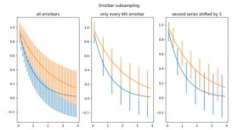
Bar chart with labels



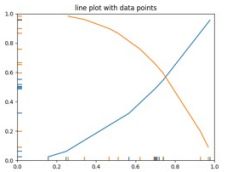
Stacked bar chart



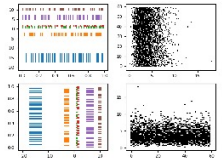
Errorbar limit selection



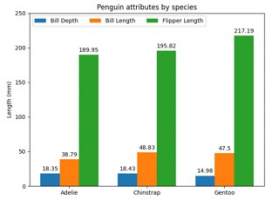
Errorbar subsampling



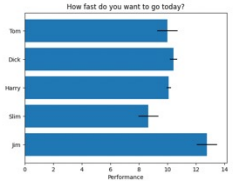
EventCollection Demo



Eventplot demo



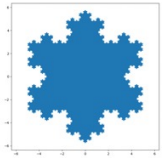
Grouped bar chart with labels



Horizontal bar chart

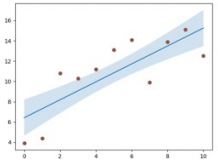
broken_barh creates sequences of horizontal bars. This example shows a timing diagram.

CapStyle

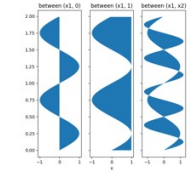


Filled polygon

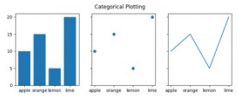
fill_between with transparency



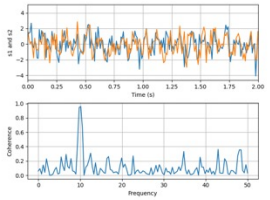
Fill the area between two lines



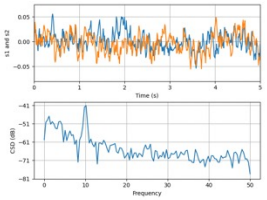
Fill the area between two vertical lines



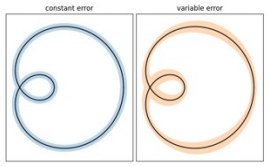
Plotting categorical variables



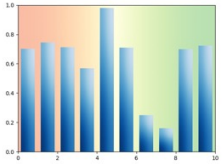
Plotting the coherence of two signals



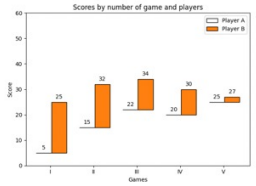
Cross spectral density (CSD)



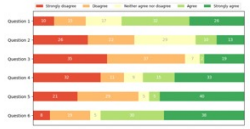
Curve with error band



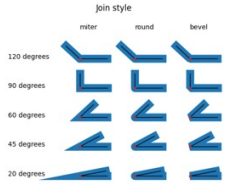
Bar chart with gradients



Hat graph



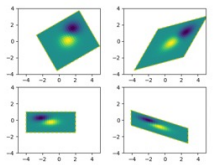
Discrete distribution as horizontal bar chart



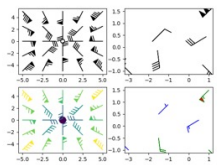
JoinStyle

Some examples

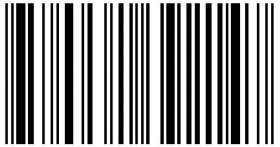
Images, contours and fields



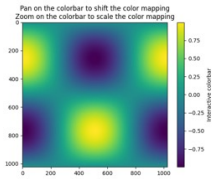
Affine transform of an image



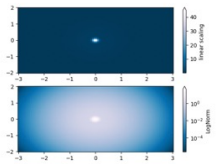
Wind barbs



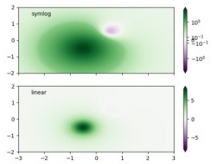
Barcode



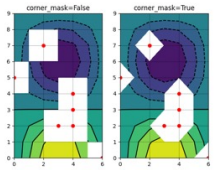
Interactive adjustment of colormap range



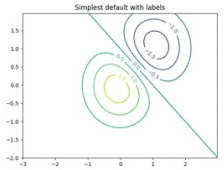
Colormap normalizations



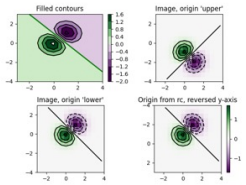
Colormap normalizations SymLogNorm



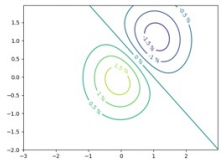
Contour corner mask



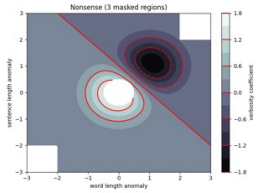
Contour Demo



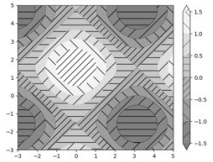
Contour image



Contour Label Demo

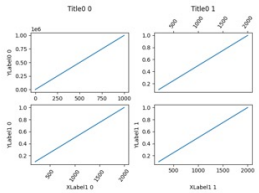


Contourf demo

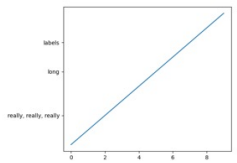


Contourf hatching

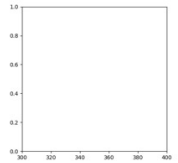
Subplots, axes and figures



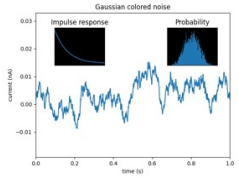
Align labels and titles



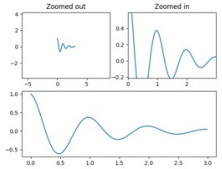
Programmatically control subplot adjustment



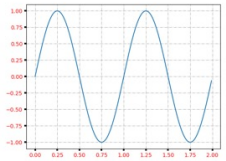
Axes box aspect



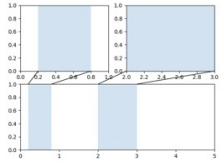
Axes Demo



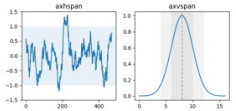
Controlling view limits using margins and sticky_edges



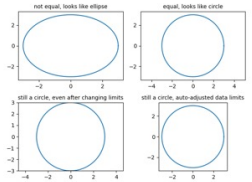
Axes properties



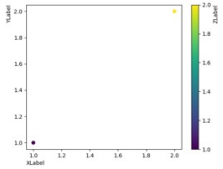
Axes zoom effect



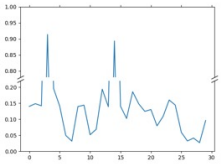
Draw regions that span an Axes



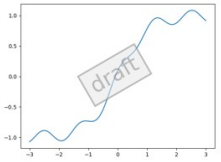
Equal axis aspect ratio



Axis label position



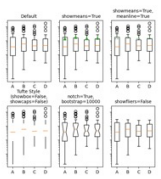
Broken axis



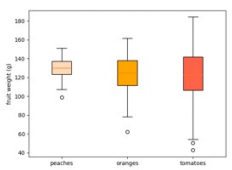
Custom Figure subclasses

Some examples

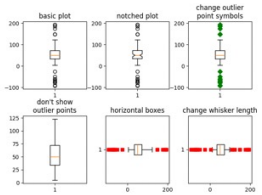
Statistics



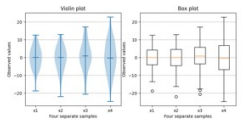
Artist customization in box plots



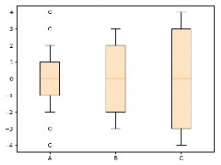
Box plots with custom fill colors



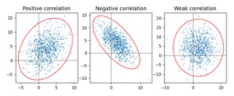
Boxplots



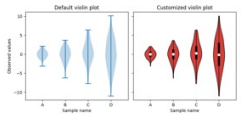
Box plot vs. violin plot comparison



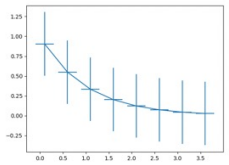
Separate calculation and plotting of boxplots



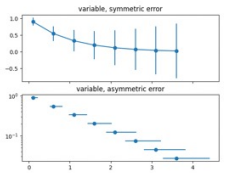
Plot a confidence ellipse of a two-dimensional dataset



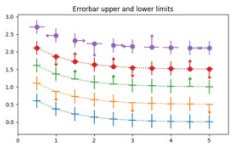
Violin plot customization



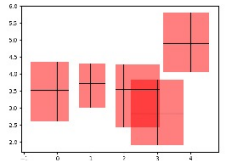
Errorbar function



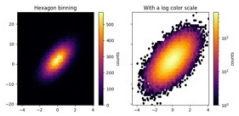
Different ways of specifying error bars



Including upper and lower limits in error bars

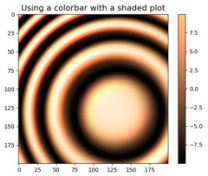


Create boxes from error bars using PatchCollection

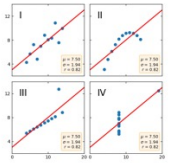


Hexagonal binned plot

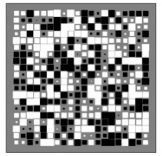
Specialty plots



Hillshading

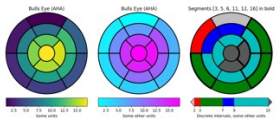


Anscombe's quartet

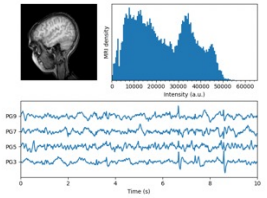


Hinton diagrams

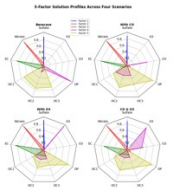
Ishikawa Diagram



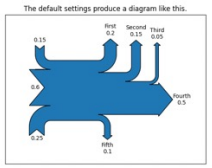
Left ventricle bullseye



MRI with EEG

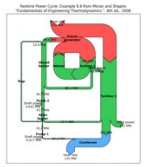


Radar chart (aka spider or star chart)

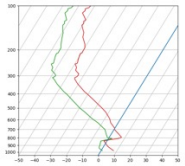


The Sankey class

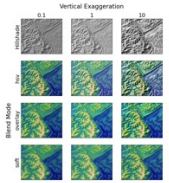
Long chain of connections using Sankey



Rankine power cycle



SkewT-logP diagram: using transforms and custom projections



Topographic hillshading

Quick aside on Python packages

- Packages are collections of modules which are themselves collections of functions.
- Can be used to add functionality or special cases to 'base' Python.
- Widely available online (also, they're free!).

NumPy

matplotlib

Pandas

scikit learn

Seaborn

Data visualisation

Matplotlib is probably the most used Python package for 2D graphics.

Before using, remember to import modules:

```
import matplotlib as plt
import numpy as np
```

More usage about matplotlib:

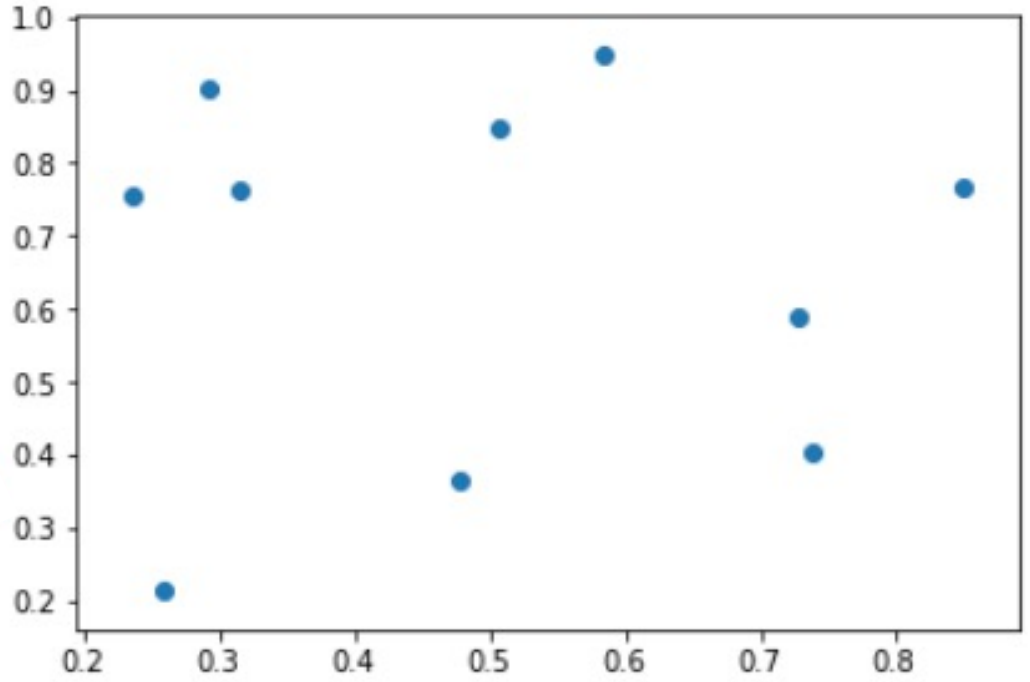
https://matplotlib.org/gallery/index.html
http://www.labri.fr/perso/nrougier/teaching/matplotlib/

Scatter plots and bar graphs

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
x = np.random.rand(N)
y = np.random.rand(N)
#Create an array of N random numbers over [0, 1)
plt.scatter(x, y, marker='o')
```

we will talk about arrays
in the next lecture

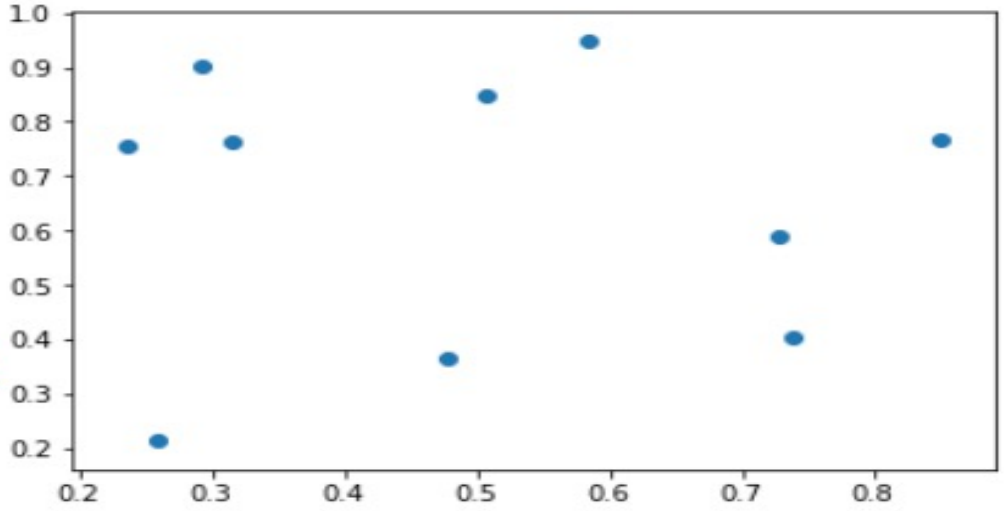


Scatter plots and bar graphs

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
x = np.random.rand(N)
y = np.random.rand(N)
#Create an array of N random numbers over [0, 1)
plt.scatter(x, y, marker='o')
```

we will talk about arrays
in the next lecture

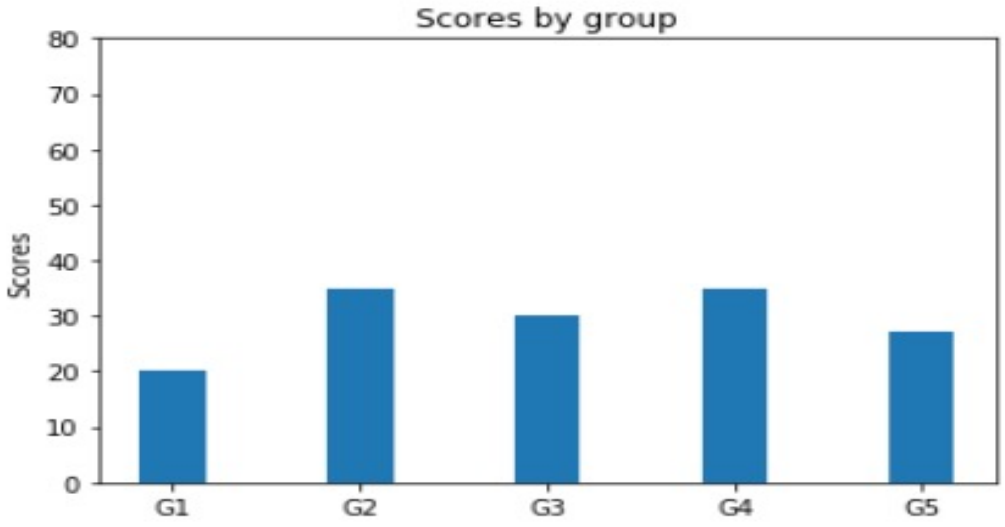


np.arange:[start,stop,step)
np.linspace:[start,stop,Num_of_Ele]

```
import numpy as np
import matplotlib.pyplot as plt

N = 5
scores = (20, 35, 30, 35, 27)
Std = (2, 3, 4, 1, 2)
ind = np.arange(N)     # the x locations for the groups
width = 0.35        # the width of the bars: can also be len(x) sequence
p1 = plt.bar(ind, scores, width, yerr=Std)
plt.ylabel('Scores')
plt.title('Scores by group')
plt.xticks(ind, ('G1', 'G2', 'G3', 'G4', 'G5'))
plt.yticks(np.arange(0, 81, 10))

plt.show()
```

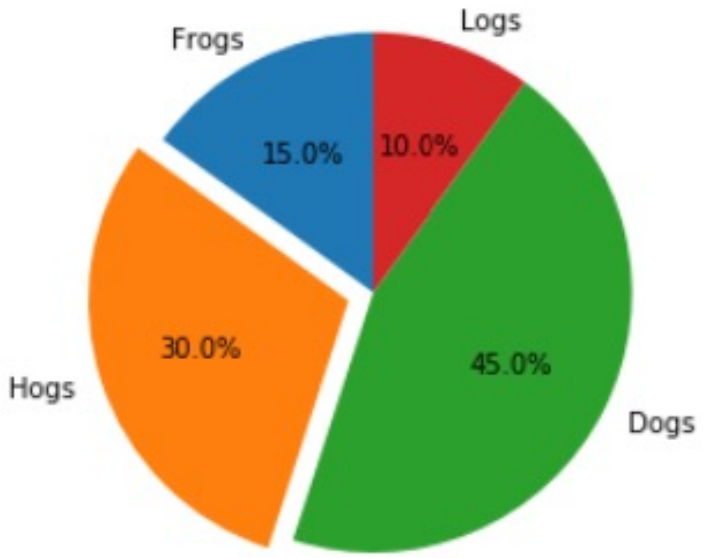


Pie charts and box plots

```
import matplotlib.pyplot as plt

# Pie chart, where the slices will be ordered and plotted counter-clockwise:
labels = 'Frogs', 'Hogs', 'Dogs', 'Logs'
sizes = [15, 30, 45, 10]
explode = (0, 0.1, 0, 0)
#specifies the fraction of the radius with which to offset each wedge
plt.pie(sizes,explode=explode, labels=labels, autopct='%1.1f%%',
        shadow=False, startangle=90)
plt.axis('equal')
# Equal aspect ratio ensures that pie is drawn as a circle.
plt.show()
```

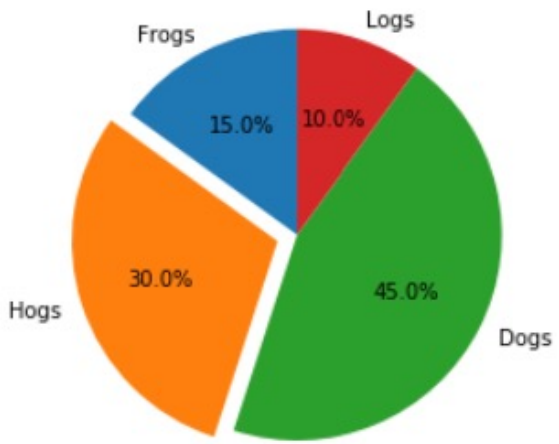
One integer and one decimal place



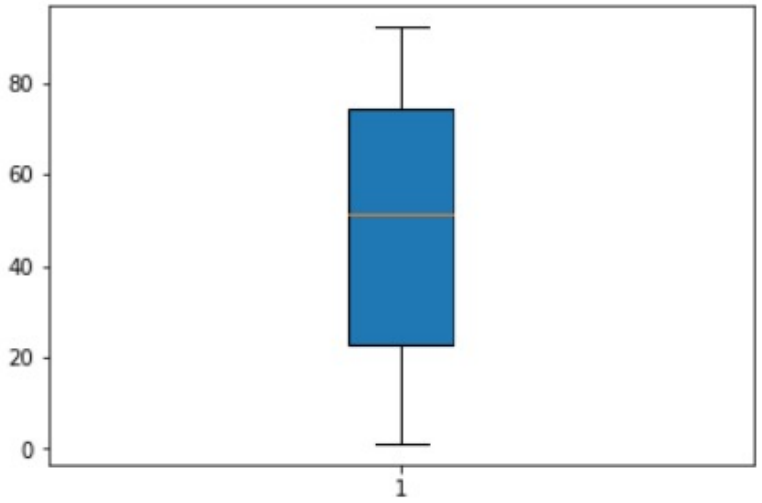
Pie charts and box plots

```
import matplotlib.pyplot as plt

# Pie chart, where the slices will be ordered and plotted counter-clockwise:
labels = 'Frogs', 'Hogs', 'Dogs', 'Logs'
sizes = [15, 30, 45, 10]
explode = (0, 0.1, 0, 0)
#specifies the fraction of the radius with which to offset each wedge
plt.pie(sizes,explode=explode, labels=labels, autopct='%1.1f%%',
        shadow=False, startangle=90)
plt.axis('equal')
# Equal aspect ratio ensures that pie is drawn as a circle.
plt.show()
```

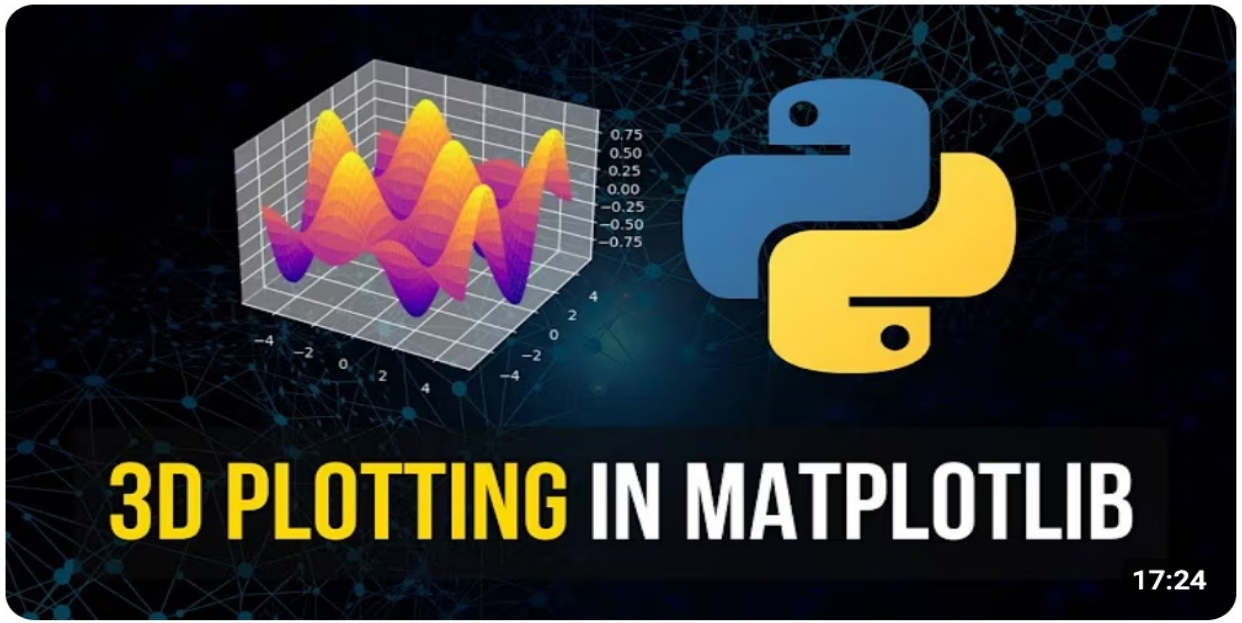


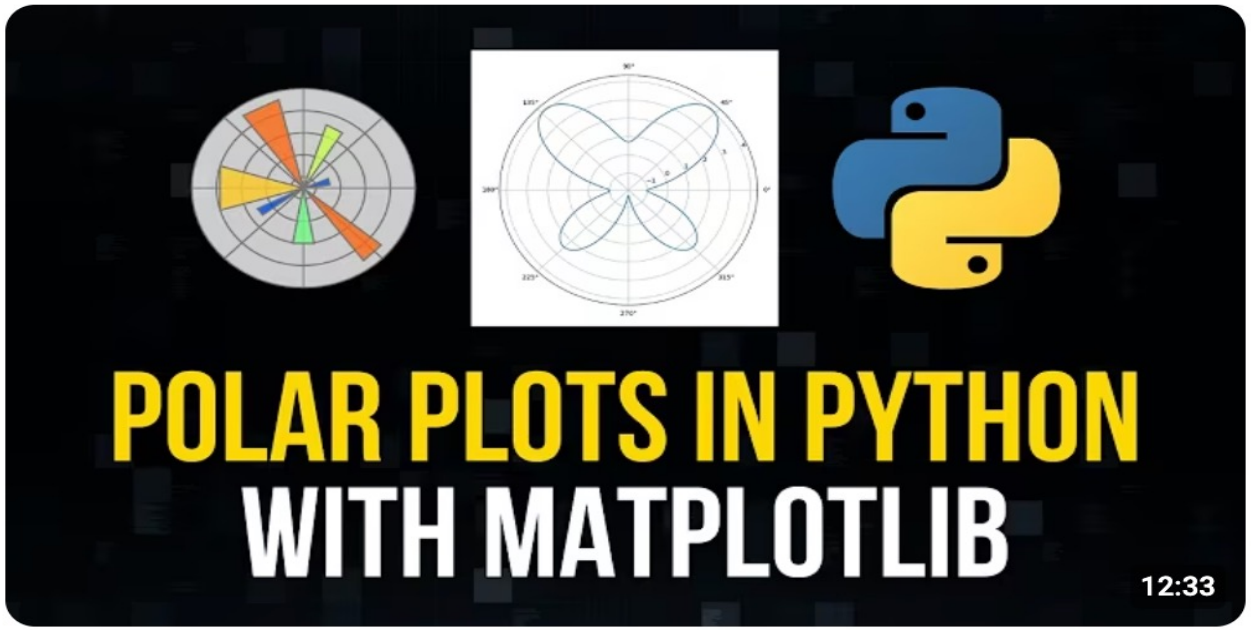

```
import numpy as np
import matplotlib.pyplot as plt
n = 20
score = np.random.uniform(0,100,n)
plt.boxplot(score,
            vert = True,
            whis = 1.5,
            patch_artist = True,
            meanline = False,
            showbox = True,
            showcaps = True,
            showfliers = True,
            notch = False
              )
plt.show()
```

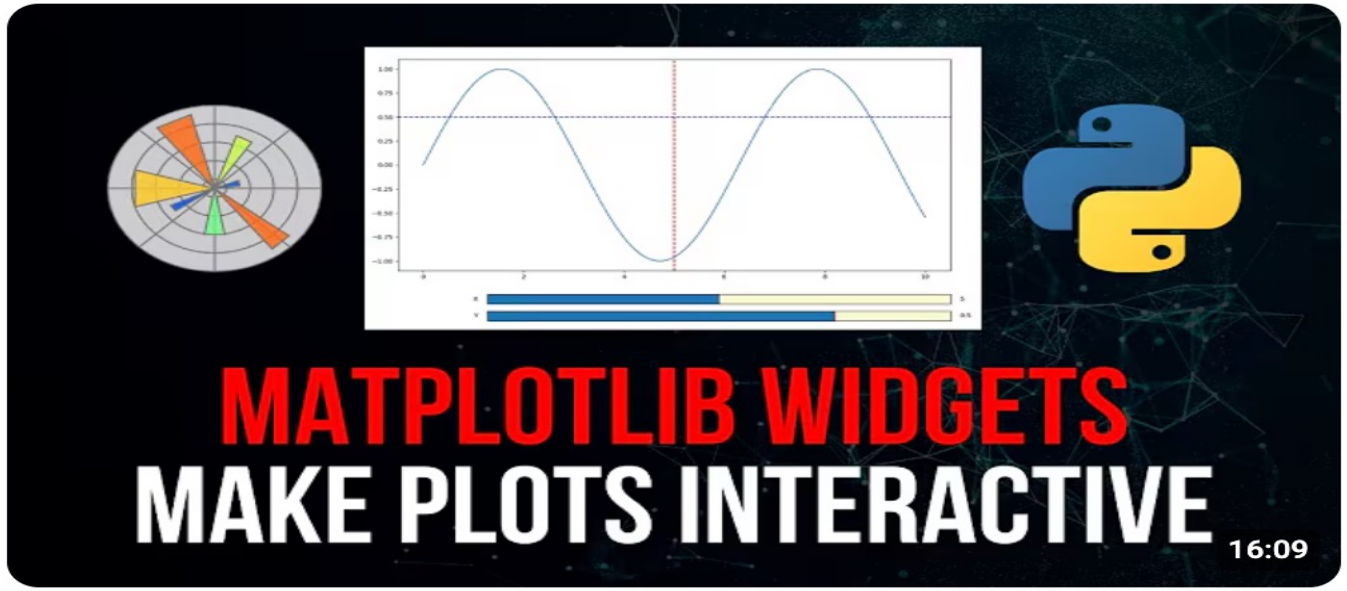


More to explore

YouTube







Learning objectives

After this lecture, you should be able to:

- Name and describe data types (revision).
- Manipulate numbers in Python.
- Visualise and summarise data in Python.

浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

Python II: Working with Numbers

Basic Data Types

Any questions?

IBI 1, Lecture 6.1

Ying CHI – yingchi@intl.zju.edu.cn

Semester 2, 2024/25